

Project Interim Report
On
Student Grading System
(Using Object-Oriented Programming)

RU-100-01-00012
Object Oriented Programming with Java

SESSION: 2025-26



Submitted By

Anmol Tamboli

RU -25 - 10227

Bachelor of Technology in Computer Science & Engineering

in association with Google

Under the Guidance of
Mr. Shivansh Mehta

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

TABLE OF CONTENTS

[illegible]

10.	Conclusion	18
11..	References	20

Abstract

The Student Grading System is a Java-based application developed using Object-Oriented Programming (OOP) principles to simplify and automate the process of student performance evaluation. In traditional grading systems, manual calculation of marks and grade assignment can be time-consuming and prone to errors. This project aims to overcome these limitations by providing an efficient and reliable solution.

The system allows users to input student details such as name and marks obtained in different subjects. It then calculates the average marks and assigns a grade based on predefined criteria. The application ensures accuracy, consistency, and reduces manual effort in the grading process.

This project demonstrates the practical implementation of core OOP concepts such as classes, objects, encapsulation, and methods. The use of a structured programming approach enhances code readability, maintainability, and reusability.

Overall, the Student Grading System serves as a simple yet effective tool for managing academic records and evaluating student performance. It can be further enhanced by integrating advanced features such as database management and graphical user interfaces for real-world applications.

Introduction

In the modern education system, the management and evaluation of student academic performance play a crucial role in ensuring quality learning outcomes. Traditional methods of grading, which involve manual calculations and record-keeping, are often time-consuming, error-prone, and inefficient. With the advancement of technology, there is a growing need for automated systems that can handle such tasks accurately and efficiently.

The Student Grading System is a software-based solution developed using Java programming language and Object-Oriented Programming (OOP) principles. This system is designed to simplify the process of storing student information, calculating average marks, and assigning grades based on predefined criteria. By automating these operations, the system reduces human effort and minimizes the chances of computational errors.

The project focuses on implementing core OOP concepts such as classes, objects, encapsulation, and methods, which help in organizing the code in a structured and reusable manner. A dedicated class is used to represent each student, containing attributes like name and marks, along with methods to perform operations such as average calculation and grade assignment.

Additionally, the system enhances clarity and consistency in grading by applying standard conditions for grade allocation. It provides a clear and organized output displaying student details, marks, average score, and final grade. This not only improves accuracy but also makes the evaluation process transparent and easy to understand.

2.1 Objectives

The primary objectives of the Student Grading System are as follows:

- To design and develop an efficient system for managing student academic records.
- To store and organize student details such as name and marks in a structured manner.
- To automate the calculation of average marks to reduce manual effort and errors.
- To assign grades based on predefined performance criteria using logical conditions.
- To implement core Object-Oriented Programming (OOP) concepts such as classes, objects, encapsulation, and methods.
- To improve accuracy, consistency, and transparency in the grading process.
- To enhance code readability, reusability, and maintainability through proper program structure.
- To provide a simple and user-friendly system for evaluating student performance.

Scope

The Student Grading System is designed to automate the process of evaluating student performance by calculating average marks and assigning grades based on predefined criteria. The scope of this project is focused on developing a simple, efficient, and reliable application using Object-Oriented Programming (OOP) concepts in Java.

The system is suitable for small-scale academic environments such as schools, coaching institutes, and for educational learning purposes. It allows users to enter student details and marks for multiple subjects, process the data, and generate accurate results in a structured format.

The current scope of the project includes handling a single student record at a time, performing average calculations, applying grading logic, and displaying the final result through a console-based interface. It emphasizes the practical implementation of OOP principles like classes, objects, encapsulation, and methods.

Although the system is basic, it has significant potential for future expansion. The project can be extended by adding features such as managing multiple student records, integrating a database for data storage, developing a graphical user interface (GUI), and generating detailed reports.

Overall, the scope of this project is to provide a foundational system that demonstrates how programming can be used to solve real-world problems in academic management, while also allowing further enhancements for advanced applications.

System Requirements

1. Hardware Requirements

<u>Components</u>	<u>Specifications</u>
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	At least 500 MB free Space
Input Devices	Keyboard
Output Devices	Monitor

2. Software Requirements

<u>Components</u>	<u>Specifications</u>
Operating System	Window / Linux / macOS
Programming Language	Java (JDK 8 or above)
IDE / Editor	VS Code / IntelliJ IDEA
Compiler	Java Compiler (javac)
Runtime	Java Runtime Environment (JRE)

3. User Requirements

<u>Requirement Type</u>	<u>Specifications</u>
Technical Skills	Intel i3 or above
RAM	Minimum 4 GB
Storage	At least 500 MB free Space

“The system requirements are minimal, making the application lightweight, cost-effective, and easily deployable on standard computing systems.”

Methodology

The Student Grading System is developed using a systematic and structured approach to ensure efficient processing of student data and accurate evaluation of academic performance. The methodology focuses on dividing the entire process into logical steps, making the system easy to understand, implement, and maintain.

The process begins with input collection, where the user enters the student's name and marks for different subjects. This data is then stored within a class structure using appropriate variables, ensuring proper organization and encapsulation.

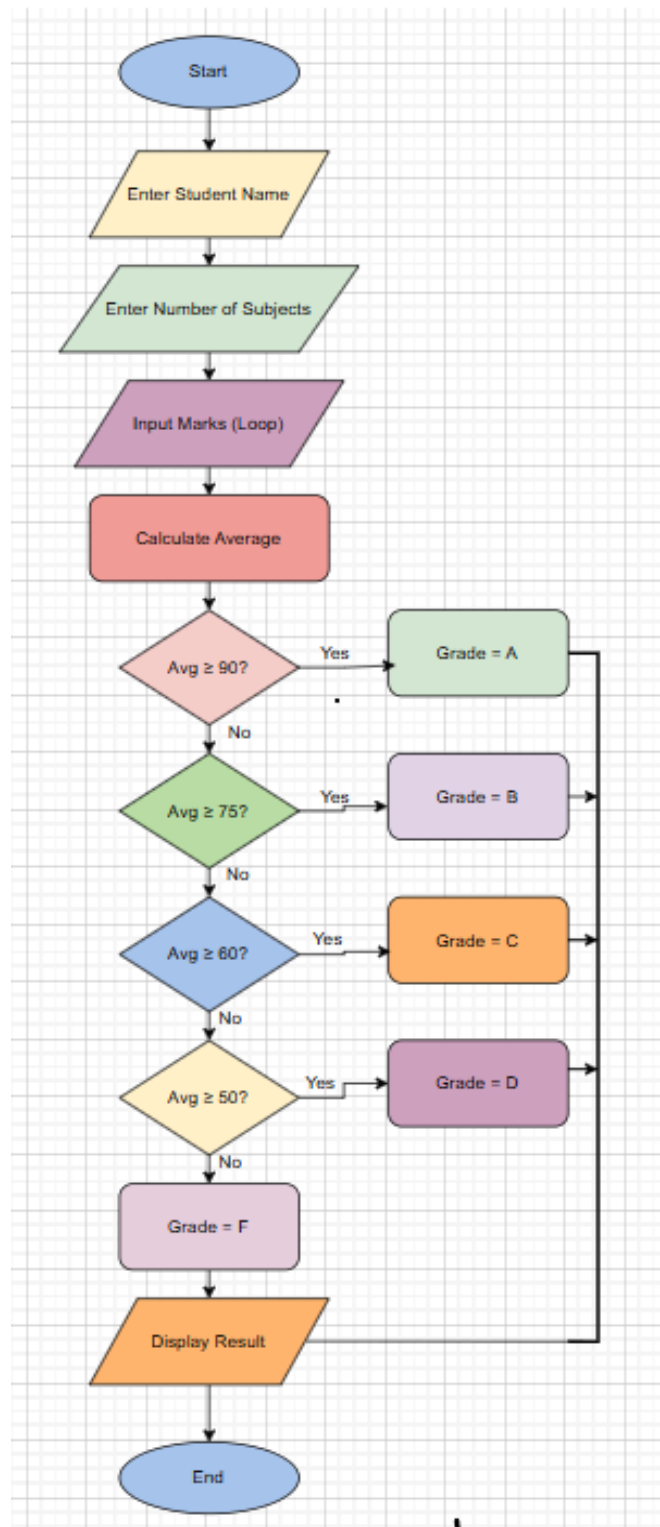
Next, an object of the Student class is created, which represents a real-world student entity. The system then performs the average calculation by summing all the marks and dividing them by the total number of subjects. This operation is carried out using a dedicated method to maintain modularity.

After calculating the average, the system proceeds to the grade assignment phase. In this step, conditional statements are used to determine the appropriate grade based on predefined criteria. This ensures consistency and fairness in evaluation.

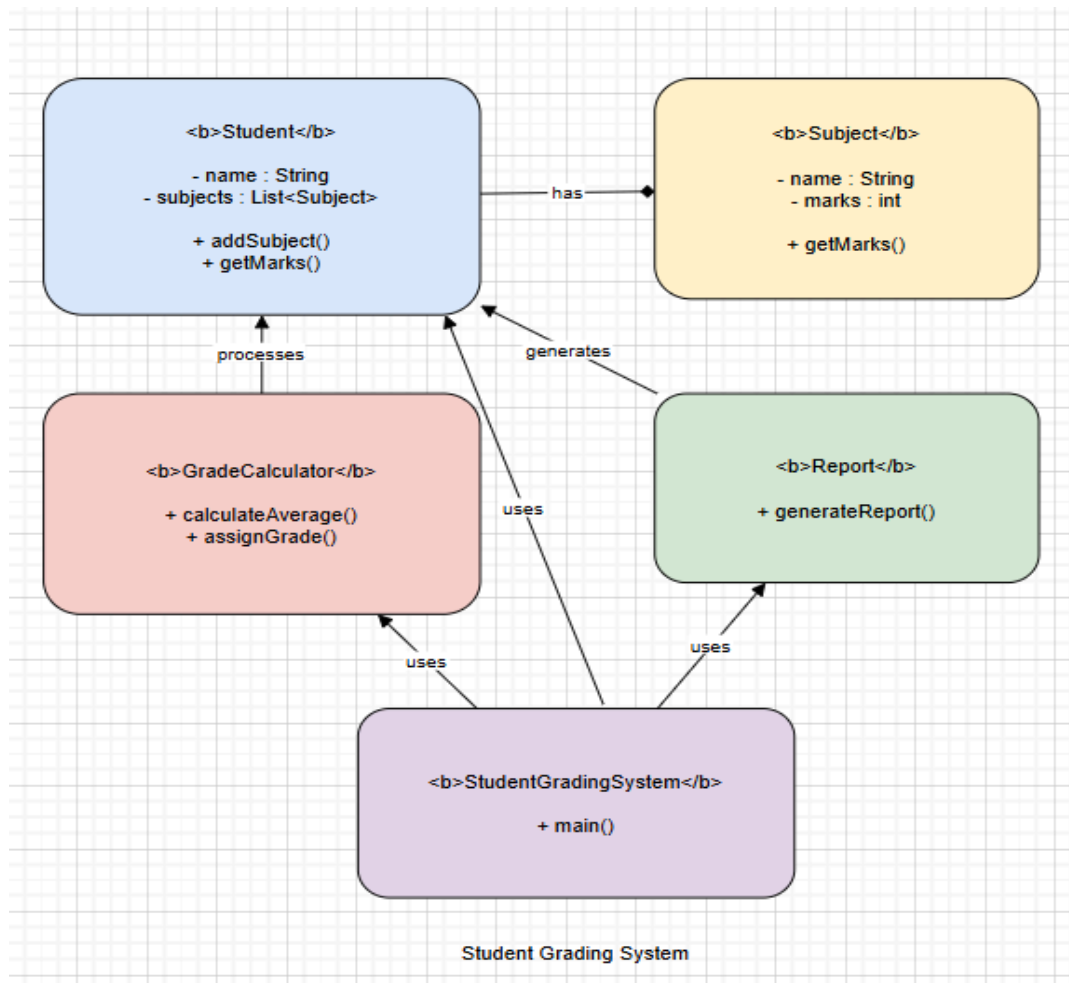
Once the grade is assigned, the system processes the final result and displays all relevant information, including student name, marks, average score, and grade, in a clear and structured format.

Overall, the methodology follows a modular and object-oriented approach, where each task is handled by a separate method. This improves code readability, reusability, and maintainability, while also demonstrating the practical implementation of Object-Oriented Programming concepts .

Flowchart



Class Diagram



Implementation

The Student Grading System is implemented using Java programming language by applying Object-Oriented Programming (OOP) principles. The system is designed in a structured and modular manner, where each functionality is divided into separate components to improve clarity, maintainability, and scalability. The implementation focuses on simulating a real-world student evaluation process through a class-based approach.

7.1 Class Structure and Design

The entire system is built around a central class named **Student**, which represents a real-world student entity. This class encapsulates both the data (attributes) and the operations (methods) required to process student information.

- **Attributes (Data Members):**
 - **name**: Stores the name of the student as a string value.
 - **marks[]**: An array used to store marks obtained in multiple subjects.

Encapsulation is applied by keeping these variables private and accessing them through methods, ensuring data security and controlled access.

7.2 Method Implementation

The functionality of the system is divided into multiple methods, each responsible for a specific task:

- **calculateAverage():**

This method calculates the average marks by summing all subject marks and dividing by the total number of subjects. It ensures accurate computation and avoids manual calculation errors.

- **assignGrade():**
This method uses conditional statements (if-else) to determine the grade based on the calculated average. It follows predefined grading criteria, ensuring consistency and fairness.
 - **displayDetails():**
This method displays all relevant student information, including name, marks, average score, and assigned grade in a structured format.
-

7.3 Program Execution Flow

The system follows a step-by-step execution process:

1. The program begins by prompting the user to enter student details such as name and marks.
2. The input data is stored within the Student class variables.
3. An object of the Student class is created to represent the student.
4. The system calls the `calculateAverage()` method to compute the average marks.
5. Based on the average, the `assignGrade()` method evaluates and assigns the appropriate grade.
6. Finally, the `displayDetails()` method outputs the complete result to the user.

This sequential flow ensures smooth execution and logical processing of data.

7.4 Grading Logic

The grading system is implemented using a set of predefined conditions:

- **Average $\geq 90 \rightarrow$ Grade A**
- **Average ≥ 75 and $< 90 \rightarrow$ Grade B**
- **Average ≥ 60 and $< 75 \rightarrow$ Grade C**
- **Average ≥ 50 and $< 60 \rightarrow$ Grade D**
- **Average $< 50 \rightarrow$ Grade F**

This logical structure ensures that each student is graded accurately based on performance.

7.5 User Interaction and Interface

The system uses a console-based interface for user interaction. The user inputs data using the keyboard, and the program displays the results on the screen. The interface is simple, user-friendly, and suitable for beginners.

The step-by-step interaction improves usability and ensures that even users with basic knowledge can operate the system effectively.

7.6 Error Handling and Validation (Optional Enhancement)

Although basic, the system can be extended to include input validation such as:

- Ensuring marks are within valid range (0–100)
- Handling incorrect data input
- Preventing invalid calculations

This enhances reliability and robustness of the system.

7.7 Scalability and Extension

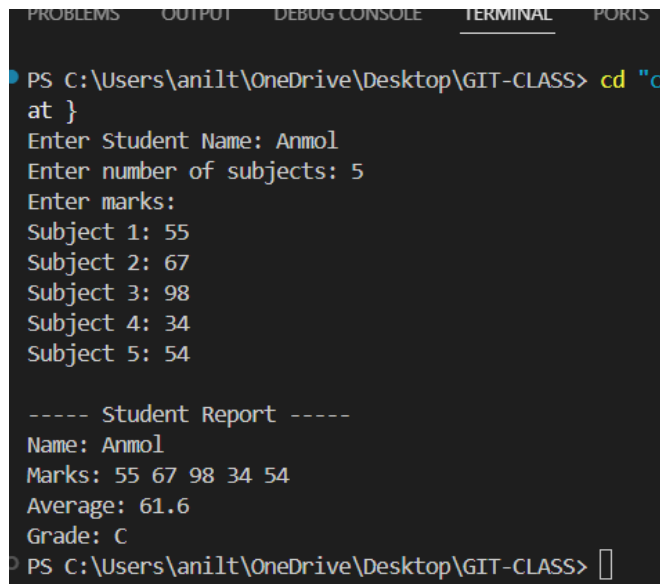
The current implementation is designed for a single student, but it can be extended to handle multiple students by using arrays or collections. Additional features such as database integration, graphical user interface (GUI), and report generation can further enhance the system.

Sample Input / Output

Enter Student Name: Rahul
Enter number of subjects: 4
Enter marks:
Subject 1: 85
Subject 2: 78
Subject 3: 90
Subject 4: 88

----- Student Report -----

Name: Rahul
Marks: 85 78 90 88
Average: 85.25
Grade: B



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\anilt\OneDrive\Desktop\GIT-CLASS> cd "c
at }
Enter Student Name: Anmol
Enter number of subjects: 5
Enter marks:
Subject 1: 55
Subject 2: 67
Subject 3: 98
Subject 4: 34
Subject 5: 54

----- Student Report -----
Name: Anmol
Marks: 55 67 98 34 54
Average: 61.6
Grade: C
PS C:\Users\anilt\OneDrive\Desktop\GIT-CLASS> 
```

Summary

The implementation of the Student Grading System effectively demonstrates the practical application of OOP concepts in Java. The modular design, clear logic flow, and structured approach make the system efficient, easy to understand, and adaptable for future improvements. It serves as a strong foundation for developing more advanced academic management systems.

Future Enhancements

To evolve this application from a basic console exercise to a production-grade academic management ecosystem, the following improvements are planned:

- **Persistent Storage (Database Integration):** Replace temporary in-memory arrays with persistent database solutions using JDBC alongside an MySQL or PostgreSQL relational engine.
- **Graphical User Interface (GUI):** Implement a modern desktop presentation layer using JavaFX or Swing components to replace command-line interface prompts with interactive, user-friendly forms.
- **Batch Data Operations & File I/O:** Allow teachers to upload **.CSV** or Excel spreadsheet gradebooks directly to execute calculations for thousands of records simultaneously.
- **Security Architecture & User Authentication:** Implement Role-Based Access Control (RBAC) separating administrative privileges (Teachers entering/editing marks) from read-only records inspection accounts (Students/Parents).
- **Analytical Dashboards:** Build analytical reporting engines using statistical calculations to calculate Class Averages, Medians, Standard Deviations, and generate charts depicting student performance trajectories over time.

Gantt Chart

W1: Requirement Analysis & Design
W2: Core Architecture Architecture (OOP Models)
W3: Business Logic Implementation (Algorithms)
W4: UI Development & Validation Frameworks
W5: Testing, QA & Integration Analysis
W6: Deployment & Final Presentation Documentation

Activity	W1	W2	W3	W4	W5	W6
----- ----- ----- ----- ----- -----						
Requirement Analysis & Specs	XXXX					
System Architecture Design		XXXX				
Core OOP Logic Engineering			XXXX			
User Input & Data Validation			XXXX			
Debugging & Defect Fixing				XXXX		
Project Documentation Signoff					XXXX	

Conclusion

The Student Grading System represents a practical and efficient approach to automating academic evaluation using Object-Oriented Programming principles. By transforming a traditionally manual process into a structured digital solution, the project highlights the importance of technology in improving accuracy, consistency, and efficiency in educational systems.

One of the key strengths of this system is its modular design, where each functionality—such as data input, average calculation, and grade assignment—is handled through well-defined methods. This not only simplifies the overall logic but also enhances code maintainability and scalability. The use of OOP concepts like encapsulation and abstraction ensures that the system is organized, reusable, and easy to extend.

From a user perspective, the system provides a clear and interactive flow: input is processed step-by-step, evaluated logically, and displayed in an understandable format. This structured interaction improves usability and makes the system intuitive even for beginners.

Although the current implementation is basic and console-based, it lays a strong foundation for future innovations. The system can be transformed into a full-fledged application by integrating graphical user interfaces, database connectivity, and analytical features such as performance tracking and report generation.

In conclusion, the project not only fulfills its primary objective of automating student grading but also demonstrates how simple programming concepts can be applied to build meaningful and scalable real-world solutions. It reflects the potential of software development in enhancing educational processes and encourages further exploration and innovation in this domain.

In addition to its functional capabilities, the system offers several important benefits:

- Improves accuracy and consistency in grading
- Reduces time required for evaluation
- Provides a clear and organized output
- Demonstrates real-world application of programming concepts

Despite its advantages, the system has certain limitations:

- It handles only one student record at a time
- It does not include database integration
- It is limited to a console-based interface

However, these limitations open opportunities for future improvements. The system can be enhanced by adding features such as graphical user interface (GUI), database connectivity, multi-user support, and advanced reporting functionalities.

Overall, the Student Grading System is a simple yet effective solution that highlights the importance of automation in education. It not only fulfills its primary objective but also provides a strong foundation for developing more advanced and scalable academic management systems.

References

Book

1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

[2] E. Balagurusamy, *Programming with Java*, 5th ed. New Delhi: McGraw-Hill Education, 2019.

Websites

[3] Oracle, “Java Documentation,” Oracle. [Online]. Available: <https://docs.oracle.com>

[4] GeeksforGeeks, “Java Programming Language,” [Online]. Available: <https://www.geeksforgeeks.org>